

```
static void param_exit(void) {
    printk(KERN_ALERT "Exiting the parameter demo");
}

module_init(param_init);
module_exit(param_exit);
```

Here's an explanation of the code that's different from the *Hello World* module:

1. All module parameter-related functions are in the `moduleparam` header file.
2. Here, `module_param(parameter name, data type, permission)` is used to tell the kernel the name of the parameter, its variable data type, and the permissions for it. Permissions are generally of five types: `S_IWUSR`, `S_IRUSR`, `S_IXUSR`, `S_IRGRP`, and `S_IWGRP`. `S_I` is a common prefix; `R` corresponds to read, `W` to write and `X` to execute. `USR` means the user, and `GRP`, the group. Permission in `module_param` can be a single permission or multiple ones using the `|` OR operator in between, as shown in the code.

Displaying the value of the parameter in the initialise function

Change *hello.o* to *parameter.o* in the *Makefile*; compile using *make*. Pass the parameter to the module during its insertion into the kernel space. Run `sudo insmod parameter.ko paramTest=2`. Please note that the name is case-sensitive. After inserting, check the kernel log. You will see the output 'Value of paramTest is:2'. If you don't specify a value for the parameter during insertion, it will take the data type's default value—0 for *int*. See Figure 2.

Passing a series of parameters

You can pass more than one parameter to the module using the concept of the arrays. Here's the code (*parameterArray.c*):

```
#include<linux/init.h>
#include<linux/module.h>
#include<linux/moduleparam.h>

MODULE_LICENSE("GPL");

int paramArray[3]; #1
module_param_array(paramArray, int, NULL, S_IWUSR|S_IRUSR); #2

static int array_init(void) {
    printk("Into the Parameter Array Demo");
    printk("Array Elements are: %d\t%d\t%d", paramArray[0],
    paramArray[1], paramArray[2]);
    return 0;
}

static void array_exit(void) {
    printk("Exiting the Array Parameter Demo");
```

```
mk@ubuntu:~/linux/module$ sudo insmod parameter.ko paramTest=2
mk@ubuntu:~/linux/module$ sudo rmmod parameter.ko
[30562.334522] Showing the parameter Demo
[30562.334533] value of the paramTest is : 2
mk@ubuntu:~/linux/module$ sudo insmod parameter.ko
mk@ubuntu:~/linux/module$ sudo rmmod parameter.ko
[30562.382211] Showing the parameter Demo
[30562.382218] value of the paramTest is : 0
mk@ubuntu:~/linux/module$ sudo rmmod parameter.ko
```

Figure 2: Passing a single parameter with module

```
41402.178040] Process Name: chrome PID:12022 Process State:1
41402.178052] Process Name: chrome PID:12071 Process State:1
41402.178063] Process Name: chrome PID:12180 Process State:1
41402.178073] Process Name: chrome PID:12210 Process State:1
41402.178083] Process Name: chrome PID:12269 Process State:1
41402.178093] Process Name: GoogleTalkL... PID:12274 Process State:1
41402.178103] Process Name: /usr/lib/tar... PID:22884 Process State:1
41402.178113] Process Name: gnome-gty-hel... PID:12241 Process State:1
41402.178123] Process Name: bash PID:12242 Process State:1
41402.178133] Process Name: udevd PID:15646 Process State:1
41402.178143] Process Name: udevd PID:21343 Process State:1
41402.178153] Process Name: kio_file PID:15649 Process State:1
41402.178163] Process Name: chrome PID:21311 Process State:1
41402.178173] Process Name: chrome PID:12159 Process State:1
41402.178183] Process Name: chrome PID:31307 Process State:1
41402.178193] Process Name: cron PID:15650 Process State:1
41402.178203] Process Name: sh PID:5008 Process State:1
```

Figure 3: Retrieving process information

```
}
```

```
module_init(array_init);
module_exit(array_exit);
```

1. Declaring an *int* array of size 3.
2. To make the array variable act as parameters, use the `module_param_array()` function instead of `module_param()`. Arguments to the function are the variable name, data type, counter and permissions, respectively. What is a counter? It is used to track the number of parameters passed to the module. In your option, you could also ignore the count and pass `NULL` as I did.

Compile the module. To insert, run `sudo insmod parameterArray.ko paramArray=1,2,3`. To verify, check the kernel log and you will see *paramArray* values as 1,2,3. If parameters are missing, the kernel again takes the default value of the data type, 0 for *int*.

Playing with processes

Each process in Linux has certain properties associated with it, such as process ID, runnable state, flags, etc, in a structure present in the *sched.h* header file in */include/linux*. The structure is named 'task_struct'. You can retrieve lots of real-time information by accessing this structure; all you need is to create a pointer to the structure, and then start using it.

I want to retrieve the process ID, process name and its state in a real-time environment; here's the code (*process.c*):

```
#include<linux/init.h>
#include<linux/module.h>
#include<linux/sched.h>

MODULE_LICENSE("GPL");
```